Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

Proyecto:

Bank USAC

PONDERACIÓN: 24 pts

Tiempo estimado: 65 hrs

Índice

1. MARCO FORMATIVO 3
1.1. Valor 3
1.2. Competencia(s) 3
1.3. Habilidad(es) blandas a formar 3
2. Resultado del Aprendizaje 3
2.1. Objetivo SMART 3
3. Resumen Ejecutivo 4
4. Enunciado del Proyecto 4
4.1 Descripción del problema o necesidad a resolver 4
4.2 Alcance del proyecto 4
4.3 Recursos y herramientas a utilizar 4
4.4 Entregables 4
5. Material de apoyo 5
6. Metodología 5
7. Cronograma 6
8. Rúbrica de Calificación 6
8.1 Requisitos para optar a la calificación 6
8.2 Detalle de la Calificación 7
8.3 Comentarios Generales 8

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Confiabilidad como prioridad (Reliability First)	El equipo diseña e instrumenta el sistema para medir, monitorear y mejorar su confiabilidad mediante métricas, trazas y logs, alineado a prácticas SRE.

1.2. Competencia(s)

Implementar observabilidad en sistemas distribuidos utilizando estándares modernos (OpenTelemetry), integrando monitoreo, trazabilidad y logging para evaluar y mejorar la confiabilidad del sistema en producción.

1.3. Habilidad(es) blandas a formar

- Análisis de fallos en sistemas distribuidos
- Pensamiento orientado a confiabilidad
- Toma de decisiones basada en métricas
- Diagnóstico de problemas complejos
- Trabajo colaborativo y ownership de cada servicio (equipo de 3)
- Comunicación técnica de incidentes (postmortem blameless)
- Coordinación y negociación técnica con otro equipo (integración entre bancos)

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	**Medible (¿Cuánto?)**	**Alcanzable (¿Cómo?)**	**Realista (¿Para qué?)**	**A Tiempo (¿Cuándo?)**
Instrumentar los 5 microservicios y el API Gateway con OpenTelemetry, centralizar métricas, logs y trazas, e integrar depósitos interbancarios	100 % de servicios instrumentados; traza end-to-end del flujo Saga; 4 SLI con SLO y alertas; mínimo 3 escenarios de fallo con postmortem	OpenTelemetry Collector, Prometheus, Jaeger/Tempo, Loki/ELK y Grafana sobre el cluster de la Fase 2	Operar el sistema bancario en la nube con prácticas SRE: detectar, diagnosticar y corregir fallos con datos	4 semanas (1 al 30 de octubre)

3. Resumen Ejecutivo

En esta fase se incorpora la capacidad de observabilidad al sistema bancario distribuido desarrollado en las Fases 1 y 2, permitiendo monitorear su comportamiento en tiempo real, identificar fallos, analizar el rendimiento y medir su confiabilidad.

Se integrará OpenTelemetry para instrumentar los servicios, un OpenTelemetry Collector para recolectar la telemetría, Prometheus para métricas, Jaeger o Tempo para trazas, ELK o Loki para logs y Grafana o Kibana para visualización y alertas. Todo el stack se desplegará en la nube mediante el pipeline CI/CD y Terraform construidos en la Fase 2.

Los SLI, SLO y SLA definidos en la Fase 1 dejarán de ser solo documentación: se medirán con datos reales, se calculará el error budget y se configurarán alertas. El equipo validará todo mediante pruebas de carga y escenarios de fallo controlados, documentando cada incidente en un postmortem.

Además, cada banco se conectará con el banco de otro grupo del curso para enviar y recibir depósitos. Esto lleva las trazas, las métricas y los SLO más allá de los límites del propio sistema, hacia una dependencia externa real.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

Los sistemas distribuidos presentan complejidad operativa, dificultando la detección de fallos, análisis de rendimiento y trazabilidad de eventos.
Es necesario implementar mecanismos de observabilidad que permitan:

- Monitorear el estado del sistema
- Analizar trazas distribuidas
- Centralizar logs
- Detectar anomalías
- Medir el cumplimiento de los SLO definidos en la Fase 1 y alertar cuando estén en riesgo
- Relacionar operaciones que cruzan los límites del banco (depósitos interbancarios)

Extensión Funcional del Sistema

Customer Service

Nueva funcionalidad: Trazabilidad de ciclo de vida del cliente

El sistema deberá registrar y exponer el ciclo completo del cliente en términos de eventos observables.

Capacidades

- Registrar eventos del ciclo de vida:
  - Creación
  - Verificación (KYC)
  - Rechazo
- Asociar todos los eventos a un correlationId
- Generar logs estructurados por cada cambio de estado
- Enmascarar u omitir los datos sensibles del cliente (documento, correo, fotografía del documento, tokens JWT) en logs y atributos de trazas

Observabilidad requerida

- Métrica: número de clientes verificados vs rechazados
- Traza: flujo completo desde creación hasta verificación
- Métrica: tiempo desde la creación hasta la verificación KYC (percentil p95)

Account Service

Nueva funcionalidad: Monitoreo de uso de cuentas

El sistema deberá medir el comportamiento de las cuentas en tiempo real.

Capacidades

- Registrar operaciones sobre cuentas:
  - Débitos
  - Créditos
- Detectar patrones:
  - Alta frecuencia de transacciones: más de N operaciones sobre una misma cuenta en una ventana de M minutos. N y M deben ser configurables (ConfigMap); valores sugeridos: 10 operaciones en 5 minutos
- Emitir el evento AccountAnomalyDetected hacia el broker. Notification & Audit Service debe registrarlo y generar una notificación de nivel WARNING
- La anomalía se registra y notifica; bloquear la cuenta es opcional y, si se hace, debe justificarse en la documentación

Observabilidad requerida

- Métrica: número de transacciones por tipo de cuenta (ahorro / corriente) y por operación (débito / crédito)
- Métrica: número de anomalías detectadas
- Logs de actividad con el accountId. El detalle por cuenta se consulta en los logs, no como etiqueta de una métrica (evitar alta cardinalidad)

Transaction Service

Nueva funcionalidad: Métricas de rendimiento y estado

El sistema deberá medir el desempeño del flujo de transacciones.

Capacidades

- Medir:
  - Tiempo de procesamiento (latencia)
  - Tasa de éxito / fallo
- Registrar los estados intermedios de la Fase 2 (PENDING, APPROVED, FAILED) como spans o eventos dentro de la traza
- Asociar cada transacción a una traza distribuida

Observabilidad requerida

- Métrica: latencia del flujo de transferencia en percentiles p50, p95 y p99 (histograma, no promedio)
- Métrica: tasa de error
- Traza completa del flujo Saga, incluyendo las compensaciones

Payment Service

Nueva funcionalidad: Simulación avanzada de fallos

El sistema deberá permitir simular condiciones reales de error, ampliando la simulación de éxito, fallo y timeout de la Fase 2.

Capacidades

- Simular:
  - Fallos intermitentes con tasa configurable (%)
  - Timeout
  - Respuestas lentas con latencia configurable (ms)
- Cambiar los parámetros de simulación en tiempo de ejecución, sin redesplegar (ConfigMap o endpoint de administración protegido en el API Gateway)
- Generar eventos de fallo controlados
- Verificar que la Saga ejecute las compensaciones y que estas sean visibles en la traza

Observabilidad requerida

- Métrica: tasa de fallos externos
- Métrica: latencia del sistema externo simulado (p95)
- Logs de errores simulados

Notification & Audit Service

Nueva funcionalidad: Auditoría consultable y análisis de logs

El servicio mantiene el registro de auditoría del negocio y permite consultarlo. La recolección de logs de infraestructura no es responsabilidad de este microservicio: la realizan agentes de recolección, para no acoplar el servicio a la operación de los demás.

Capacidades

- Recolectar los logs de todos los servicios mediante un agente de infraestructura (OpenTelemetry Collector, Fluent Bit o Promtail). Los servicios no envían sus logs a este microservicio por el broker
- Clasificar eventos de auditoría y logs:
  - INFO
  - WARNING

    - ERROR
- Almacenar logs en ELK o Loki
- Exponer, a través del API Gateway, la consulta de eventos históricos por correlationId, traceId, servicio y nivel
- Generar una notificación al recibir AccountAnomalyDetected o una alerta de SLO

Observabilidad requerida

- Dashboard de logs
- Búsqueda por correlationId y traceId
- Navegación desde un log hacia su traza

Integración interbancaria (Payment, Transaction y Account Service)

Nueva funcionalidad: Depósitos entre bancos

Cada grupo será emparejado con otro grupo del curso (asignación del auxiliar) y ambos bancos deberán poder enviarse y recibir depósitos directamente. Para que cualquier pareja sea compatible, el curso publicará un contrato base (OpenAPI) al asignar el proyecto. Los grupos no deben modificarlo; solo acuerdan entre sí URLs, credenciales y horarios de prueba.

Capacidades

- Envío de depósitos al banco pareja:
    - Transaction Service inicia la Saga y Account Service reserva el monto en la cuenta origen
    - Payment Service envía la solicitud al API Gateway del banco pareja y recibe una respuesta 202 Accepted
    - El banco pareja confirma el resultado por callback: si es ACCEPTED, el débito se confirma; si es REJECTED o vence el tiempo de espera, se ejecuta la compensación y se libera el monto
- Recepción de depósitos desde el banco pareja:
    - La solicitud entra por el API Gateway, autenticada con la API key o el JWT de cliente acordado con el banco pareja (guardado en un Secret de Kubernetes)
    - Payment Service valida la cuenta destino y publica un evento; Account Service acredita el monto
    - El banco responde al banco pareja por callback con el resultado (ACCEPTED, o REJECTED con su motivo)
- Estados del depósito interbancario: PENDING, SENT, ACCEPTED, REJECTED, TIMEOUT y REVERSED
- Idempotencia obligatoria mediante idempotencyKey: un reintento del banco pareja nunca debe acreditar dos veces
- Datos mínimos del mensaje: interbankTransferId, idempotencyKey, banco origen, banco destino, cuenta destino, monto, moneda (GTQ), timestamp y traceparent
- Validaciones: cuenta destino existente y activa, cliente origen con KYC verificado (Fase 2) y monto máximo por operación configurable
- Implementar un simulador (mock) del banco pareja que cumpla el contrato, para desarrollar sin depender del otro grupo y para inyectar fallos (rechazos, timeouts y respuestas lentas)
- Frontend: enviar depósitos al banco pareja y consultar el estado de cada uno
- Acuerdo de integración: documento breve entre ambos grupos con URLs, mecanismo de autenticación, cuentas de prueba y ventana de pruebas conjuntas

- Conciliación diaria (opcional): reporte que compare los depósitos registrados con los del banco pareja

Observabilidad requerida

- Propagar el traceparent hacia y desde el banco pareja, para relacionar un depósito entre ambos bancos con el mismo traceId
- Métrica: tasa de éxito de depósitos interbancarios (enviados y recibidos)
- Métrica: latencia de confirmación del banco pareja (p95) y número de timeouts
- SLO y alerta específicos para la dependencia con el banco pareja
- Logs de cada cambio de estado del depósito, sin exponer datos sensibles

Reglas para la evaluación en parejas

- Cada grupo se califica de forma independiente: los fallos del banco pareja no restan puntos si el banco propio reacciona correctamente (compensación, reintentos y alertas)
- Si el banco pareja no está disponible el día de la calificación, el auxiliar asignará otro banco funcional para la prueba; como último recurso, se evalúa contra el mock
- Los fallos de los escenarios de prueba se inyectan con el mock propio, nunca apagando o alterando el sistema del otro grupo

Instrumentación con OpenTelemetry

Todos los servicios, incluido el API Gateway, deben:

- Generar trazas distribuidas
- Incluir:
  - traceId
  - spanId
- Propagar el contexto con el estándar W3C Trace Context (traceparent) en los headers de cada mensaje del broker (headers de Kafka, propiedades AMQP de RabbitMQ o headers de NATS), de modo que productor y consumidor queden en la misma traza
- Mantener el correlationId de la Fase 1 como atributo del span y en los logs: el traceId identifica la traza técnica y el correlationId el flujo de negocio
- Enviar la telemetría por OTLP a un OpenTelemetry Collector desplegado en el cluster
- Seguir las convenciones semánticas de OpenTelemetry para nombrar spans, métricas y atributos

Métricas (SLI base)

El sistema debe medir y almacenar en Prometheus (o una alternativa compatible):

- Latencia de transacciones en percentiles p50, p95 y p99 (histogramas, no promedios)
- Tasa de errores
- Throughput (eventos por segundo por servicio y por tópico/cola)
- Consumer lag del broker (mensajes pendientes por consumidor)
- Disponibilidad: porcentaje de solicitudes exitosas (respuestas distintas de 5xx) en el API Gateway

SLO, error budget y alertas

Tomando los SLI, SLO y SLA definidos en la Fase 1, el equipo deberá:

- Medir cada SLO con datos reales y mostrar su cumplimiento en un dashboard
- Calcular el error budget restante de cada SLO
- Configurar al menos 4 alertas (Grafana Alerting o Alertmanager), incluida al menos una por tasa de consumo del error budget (burn rate)
- Enviar las alertas a un canal visible (correo, Slack, Discord o Notification & Audit Service)
- Documentar un runbook por alerta: qué significa, cómo diagnosticarla y cómo mitigarla

Logging estructurado

Todos los logs deben:

- Estar en formato estructurado (JSON)
- Incluir:
  - timestamp
  - serviceName
  - correlationId
  - traceId y spanId
  - nivel de log
  - environment (dev / prod, según los entornos de la Fase 2)
  - version (tag de la imagen desplegada)
- Omitir o enmascarar datos sensibles: número de documento, correo, fotografías, contraseñas y tokens JWT

Dashboards obligatorios

Se deben implementar dashboards que muestren:

- Estado del sistema
- Métricas clave (SLI)
- Cumplimiento de SLO y error budget
- Errores en tiempo real
- Flujo de trazas
- Consumer lag del broker
- Uso de CPU / memoria y número de pods (HPA)
- Despliegues marcados como anotaciones

Los dashboards y las reglas de alerta deben versionarse como código (JSON / YAML) en el repositorio; no se aceptan dashboards creados solo a mano.

Integración con el sistema existente

Las nuevas funcionalidades deben integrarse con:

- Flujo de Saga existente, incluidas sus compensaciones
- Eventos del sistema
- Pipeline CI/CD de la Fase 2: el stack de observabilidad se despliega con Terraform / Helm desde el pipeline, sin pasos manuales
- Entornos dev y prod: la telemetría debe identificar el entorno
- Autoescalado (HPA) de la Fase 2: evidenciar el escalado bajo carga en los dashboards
- Frontend: mostrar el correlationId / traceId de cada operación y permitir consultar la auditoría

Pruebas de carga y escenarios de fallo

- Ejecutar pruebas de carga con k6, Locust o JMeter sobre el flujo de transferencia
- Evidenciar en los dashboards la latencia, el error rate, el throughput y la creación de pods por el HPA
- Ejecutar al menos 3 escenarios de fallo (el del banco pareja es obligatorio):
  - Fallos y timeouts del Payment Service
  - Eliminación de un pod durante la carga
  - Latencia o caída temporal del broker
  - El banco pareja no responde, responde lento o rechaza depósitos (simulado con el mock)
- Documentar cada escenario en un postmortem blameless: cronología, impacto en los SLO, evidencia (trazas, logs y métricas), causa raíz y acciones de mejora

4.2 Alcance del proyecto

Alcance obligatorio:

- Implementación de **OpenTelemetry** en los 5 microservicios y el API Gateway
- Despliegue de un **OpenTelemetry Collector** en el cluster
- Recolección de:
  - Métricas (Prometheus o compatible)
  - Logs
  - Trazas distribuidas (Jaeger o Tempo)
- Implementación de stack de logging:
  - ELK (Elasticsearch, Logstash, Kibana)
  - o Loki + Grafana
- SLO, error budget y alertas basados en las definiciones de la Fase 1
- Visualización de métricas, logs y trazas con dashboards versionados como código
- Despliegue completo en la nube mediante el pipeline de la Fase 2
- Pruebas de carga y escenarios de fallo con postmortem
- Extensión funcional del sistema y del frontend
- Integración interbancaria: envío y recepción de depósitos con el banco de otro grupo (parejas)
- Actualización de documentación

Restricciones

- Mantener arquitectura asíncrona
- No eliminar componentes previos
- No instrumentación superficial (debe ser funcional)
- No desplegar manualmente el stack de observabilidad (misma regla de la Fase 2)
- No registrar datos sensibles en logs ni en atributos de trazas

- La comunicación con el banco pareja puede usar HTTP con patrón asíncrono (202 Accepted + callback), siempre entrando por el API Gateway y saliendo desde el Payment Service. Entre los microservicios de un mismo banco la comunicación sigue siendo 100 % asíncrona por el broker

Consideraciones de costo

- Elasticsearch requiere bastante memoria; se recomienda Loki por su menor consumo de recursos
- Configurar la retención de datos (por ejemplo, 7 días) y dimensionar los recursos para no agotar los créditos de la nube

4.3 Recursos y herramientas a utilizar

Tipo	Categoría	Descripción
Obligatorio	Observabilidad	OpenTelemetry (SDK en cada servicio)
Obligatorio	Logging	ELK o Loki (se recomienda Loki por costo)
Obligatorio	Visualización	Grafana / Kibana
Obligatorio	Cloud	AWS / GCP / Azure
Obligatorio	Kubernetes	Cluster en nube
Obligatorio	Recolector de telemetría	OpenTelemetry Collector
Obligatorio	Métricas	Prometheus (o compatible: Mimir, VictoriaMetrics)
Obligatorio	Trazas	Jaeger o Grafana Tempo
Obligatorio	Alertas	Grafana Alerting o Alertmanager
Obligatorio	Pruebas de carga	k6, Locust o JMeter
Obligatorio	IaC y CI/CD	Terraform y pipeline de la Fase 2
Obligatorio	Integración interbancaria	Contrato OpenAPI base provisto por el curso; banco pareja asignado por el auxiliar

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Código	Instrumentación OpenTelemetry en los 5 microservicios y el API Gateway; nuevas funcionalidades
Infraestructura	Stack de observabilidad (Collector, Prometheus, Jaeger/Tempo, Loki/ELK, Grafana) desplegado con Terraform / Helm desde el pipeline
Dashboards	Dashboards y reglas de alerta exportados como código (JSON / YAML) en el repositorio
Documento	Arquitectura actualizada (C4 y despliegue con el stack de observabilidad), catálogo de métricas, SLI/SLO con error budget y runbooks por alerta, en markdown
Postmortems	Un postmortem blameless por cada escenario de fallo (mínimo 3)
Pruebas de carga	Scripts de carga y resultados con capturas de los dashboards
Frontend	Visualización del correlationId / traceId de cada operación y consulta de auditoría
Integración interbancaria	Conector con el banco pareja, mock del banco pareja, acuerdo de integración y evidencia de depósitos enviados y recibidos con sus trazas

Repositorio	Mismo monorepo de las fases anteriores: Proyecto-SA-<<SECCIÓN>>-<<#Grupo>>

5. Material de apoyo

Categoría (Manual / Documentación oficial/ Ejemplo / Etc)	Link	Descripción
Documentación oficial	https://opentelemetry.io/docs/	Instrumentación de observabilidad
Documentación oficial	https://grafana.com/docs/loki/latest/	Logging con Loki
Documentación oficial	https://www.elastic.co/guide/index.html	Stack ELK
Manual	https://sre.google/workbook/table-of-contents/	Prácticas SRE
Documentación oficial	https://opentelemetry.io/docs/collector/	OpenTelemetry Collector
Documentación oficial	https://opentelemetry.io/docs/specs/semconv/	Convenciones semánticas de OpenTelemetry
Documentación oficial	https://www.w3.org/TR/trace-context/	W3C Trace Context (propagación de contexto)
Documentación oficial	https://prometheus.io/docs/introduction/overview/	Métricas con Prometheus
Documentación oficial	https://www.jaegertracing.io/docs/	Trazas con Jaeger
Documentación oficial	https://grafana.com/docs/tempo/latest/	Trazas con Grafana Tempo
Documentación oficial	https://grafana.com/docs/grafana/latest/alerting/	Alertas en Grafana
Documentación oficial	https://grafana.com/docs/k6/latest/	Pruebas de carga con k6
Manual	https://sre.google/workbook/alerting-on-slos/	Alertas basadas en SLO y burn rate

Manual	https://sre.google/sre-book/postmortem-culture/	Cultura de postmortem blameless
Documentación oficial	https://spec.openapis.org/oas/latest.html	Especificación OpenAPI (contrato interbancario)
Manual	https://microservices.io/patterns/data/saga.html	Patrón Saga y compensaciones
Ejemplo	https://docs.stripe.com/api/idempotent_requests	Idempotencia con idempotency keys en pagos

6. Metodología

A continuación, se presenta una metodología de la lógica que puedes seguir para elaborar el presente proyecto:

Pasos a seguir:

1. Paso 1: Análisis del sistema desplegado
   Evaluar el sistema de la Fase 2 en la nube y revisar los SLI, SLO y SLA definidos en la Fase 1.

2. Paso 2: Diseño de la telemetría
   Definir el catálogo de métricas, los nombres de spans, los campos de log y las alertas antes de implementar.

3. Paso 3: Despliegue del stack de observabilidad
   Desplegar Collector, Prometheus, Jaeger/Tempo, Loki/ELK y Grafana con Terraform / Helm desde el pipeline.

4. Paso 4: Instrumentación con OpenTelemetry
   Agregar trazas y métricas en microservicios y Gateway, propagando el contexto por el broker.

5. Paso 5: Logging estructurado y centralizado
   Generar logs JSON con traceId y spanId, enmascarando datos sensibles.

6. Paso 6: Extensión funcional
   Implementar las nuevas funcionalidades, incluida la integración interbancaria (primero contra el mock y luego con el banco pareja), instrumentándolas desde el inicio.

7. Paso 7: SLO, alertas y dashboards
   Crear dashboards como código, calcular el error budget y configurar alertas con su runbook.

8. Paso 8: Pruebas de carga y de fallos

Ejecutar la carga y los escenarios de fallo, observando el HPA y los SLO.

9. Paso 9: Análisis de incidentes
   Redactar un postmortem blameless por escenario.

10. Paso 10: Documentación
    Actualizar arquitectura, catálogo de métricas, runbooks y manual de usuario.

Recomendaciones

- Instrumentar desde el código (no solo infraestructura)
- Correlacionar eventos con trazas
- Diseñar dashboards útiles (no decorativos)
- Validar escenarios de fallo
- Usar percentiles, no promedios, para la latencia
- Evitar etiquetas de alta cardinalidad (accountId, customerId) en las métricas
- Configurar retención de datos para controlar costos

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de proyecto	1 de octubre	1 de octubre
Elaboración	1 de octubre	23 de octubre
Calificación	24 de octubre	24 de octubre

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos que se indiquen en esta sección, de no cumplir usted tendrá una nota con valor de cero puntos.

Tema	**Descripción**	**Cumple (Sí/No)**

Despliegue en nube	Sistema desplegado y accesible en cloud (K8s)	
OpenTelemetry	Instrumentación funcional en todos los microservicios (trazas + métricas)	
Trazabilidad distribuida	Propagación de traceId / correlationId entre servicios y a través del broker	
Logging centralizado	Implementación funcional de ELK o Loki con ingestión de logs	
Logs estructurados	Logs en formato JSON (incluye timestamp, service, nivel, correlationId, traceId) sin datos sensibles	
Dashboards	Visualización de métricas y logs (Grafana/Kibana)	
Métricas SRE	Implementación de métricas: latencia en percentiles, error rate, throughput, consumer lag y disponibilidad	
Integración con Saga	Trazas y métricas integradas al flujo de transferencia (Saga)	
Nuevas funcionalidades	Funcionalidades de observabilidad implementadas en los 5 servicios	
Simulación de fallos	Implementación de fallos controlados (timeouts, errores, delays)	
Consistencia arquitectónica	Se mantiene asincronía y desacoplamiento	
Documentación actualizada	Diagramas, arquitectura y métricas actualizadas	

Sistema funcional	Flujo end-to-end observable (con trazas y logs)	
Stack de observabilidad	Collector, Prometheus, Jaeger/Tempo y Loki/ELK desplegados sin pasos manuales	
SLO y alertas	SLO de la Fase 1 medidos y al menos 4 alertas configuradas	
Postmortems	Mínimo 3 escenarios de fallo documentados	
Integración interbancaria	Depósitos enviados y recibidos con el banco pareja, con idempotencia y compensación	
UEDI	Entrega en UEDI en la fecha estipulada	
Repositorio	Commit antes de la fecha de entrega. Agregar al auxiliar como colaborador.	

8.2 Detalle de la Calificación

	Criterio/Nivel			**Nivel de evaluación**		
No.	**Criterio de evaluación**	**Punteo maximo**		**Satisfactorio 100% - 61%**	**Necesita mejorar 60% - 0%**	**Punteo Obtenido**
1	Habilidades	40				
1.1	Trabajo en equipo	10	Descripción	Distribución clara de responsabilidades, respaldada con commits de todos los integrantes y metodología ágil	Trabajo desbalanceado, sin commits que lo respalden ni evidencia de metodología ágil	
			Rango del punteo	10-6	5-0	
1.2	Documentación técnica	10	Descripción	Diagramas C4 y de despliegue actualizados con el stack de observabilidad, catálogo de métricas y runbooks coherentes con la implementación	Documentación incompleta, desactualizada o inconsistente con el sistema	

			Rango del punteo	10-6	5-0	
1.3	Análisis de fallos (postmortem)	10	Descripción	Postmortems con cronología, impacto en SLO, evidencia de trazas / logs / métricas, causa raíz y acciones	Sin postmortem o análisis sin evidencia	
			Rango del punteo	10-6	5-0	
1.4	Justificación de decisiones	10	Descripción	Stack, métricas, SLO y umbrales de alerta justificados con trade-offs (costo, retención, cardinalidad)	Sin justificación o basada en opiniones	
			Rango del punteo	10-6	5-0	
	Sub-Total de Puntos	40				
2	Conocimientos	60				

2.1	Instrumentación OpenTelemetry	6	Descripción	OpenTelemetry integrado en el código de los 5 servicios y el Gateway, enviando datos al Collector	Instrumentación parcial, incorrecta o solo de infraestructura	
			Rango del punteo	6-4	3-0	
2.2	Trazas distribuidas y propagación de contexto	8	Descripción	Flujo Saga completo en una sola traza, con contexto propagado por el broker y correlationId como atributo	Trazas cortadas entre servicios o inexistentes	
			Rango del punteo	8-5	4-0	
2.3	Logging centralizado y estructurado	6	Descripción	Logs JSON de todos los servicios, con traceId, consultables y sin datos sensibles	Logs dispersos, no estructurados o con datos sensibles	
			Rango del punteo	6-4	3-0	
2.4	Métricas (SLI)	6	Descripción	Latencia en percentiles, error rate, throughput, consumer lag y disponibilidad medidos sin etiquetas de alta cardinalidad	Métricas incompletas, basadas en promedios o que no reflejan el sistema real	

			Rango del punteo	6-4	3-0	
2.5	SLO, error budget y alertas	8	Descripción	SLO de la Fase 1 medidos, error budget calculado y alertas con runbook que se disparan durante las pruebas	Sin alertas o SLO no medidos	
			Rango del punteo	8-5	4-0	
2.6	Dashboards	6	Descripción	Dashboards útiles, versionados como código, que permiten diagnosticar un fallo	Dashboards decorativos o creados a mano sin versionar	
			Rango del punteo	6-4	3-0	
2.7	Funcionalidades nuevas	6	Descripción	Funcionalidades de los 5 servicios implementadas e integradas a la Saga, generando métricas, logs y trazas reales	Funcionalidades incompletas o sin impacto en la observabilidad	
			Rango del punteo	6-4	3-0	

2.8	Pruebas de carga y fallos	6	Descripción	Carga y 3 escenarios de fallo ejecutados, con HPA y compensaciones visibles en los dashboards	Fallos no simulados, sin carga o sin evidencia	
			Rango del punteo	6-4	3-0	
2.9	Integración interbancaria	8	Descripción	Depósitos enviados y recibidos con el banco pareja, con idempotencia, compensación ante rechazo o timeout y traza con el mismo traceId	Integración incompleta, depósitos duplicados o sin compensación	
			Rango del punteo	8-5	4-0	
	Sub-Total de Puntos	60				
	Total	**100**				

8.3 Comentarios Generales

Penalizaciones críticas

Se reducirá significativamente la nota si:

- No existen trazas distribuidas reales (el contexto no cruza el broker): 50 %
- OpenTelemetry no está integrado en el código de los servicios: 50 %

- Métricas que no reflejan el sistema real (valores fijos o simulados): 50 %
- Se rompe la comunicación asíncrona o se introduce acoplamiento entre servicios: 50 %
- Logs no centralizados: 25 %
- Dashboards que no aportan valor para diagnosticar: 25 %
- Las nuevas funcionalidades no generan observabilidad: 25 %
- Despliegue manual del stack de observabilidad: 25 %
- Datos sensibles expuestos en logs o trazas: 25 %
- Depósito interbancario duplicado o monto perdido sin compensación: 50 %